

B.Sc.CSIT 4th Semester

Operating System: CSC259

Lab-02: TO BE FAMILIAR WITH PROCESS MANAGEMENT

OBJECTIVES

- TO CREATE A SINGLE PROCESS WITH ID
- TO CREATE A CHILD AND PARENT PROCESS
- TO USE MULTIPLE FORK STATEMENT.

REQUIRED THEORY

Process management involves various tasks like creation, scheduling, termination of processes, and a dead lock. Process is a program that is under execution, which is an important part of modern-day operating systems. The OS must allocate resources that enable processes to share and exchange information. It also protects the resources of each process from other methods and allows synchronization among processes. It is the job of OS to manage all the running processes of the system. It handles operations by performing tasks like process scheduling and such as resource allocation.

REQUIRED PROGRAMS

a) To create a single process with id

```
#include<stdio.h>

#include<unistd.h>

int main() {

printf(" this is second lab");

fork();

printf(" Process is creating");

printf(" I am process %d", getpid());

return 0;

}
```

b) To create a child and parent process

```
# include<stdio.h>

#include<unistd.h>

int main() {

int pid;

printf(" This is our second lab");

printf(" I am the parent process and pid is %d", getpid());

print(" before use of forking");

pid=fork();

printf(" after forking");

if (pid == 0)

printf(" I am the child process and pid is %d", getpid());

else

printf(" I am the parent process and pid %d", getpid());

return 0; }
```

c) To use multiple fork statement.

```
# include<stdio.h>

#include<unistd.h>

int main() {

printf( " before first forking statement");

fork();

Printf("After forking statement");

fork();

printf(" After second forking");

printf(" hello world form process %d", getpid());

return 0; }
```

OUTPUT

```
ankit@ankit-virtual-machine:~$ ls
ankit  Desktop  Downloads  Pictures  snap  Videos
ankit.txt  Documents  Music  Public  Templates
ankit@ankit-virtual-machine:~$ mkdir lab02
ankit@ankit-virtual-machine:~$ cd lab02
ankit@ankit-virtual-machine:~/lab02$ touch process1.c
ankit@ankit-virtual-machine:~/lab02$ nano process1.c
ankit@ankit-virtual-machine:~/lab02$ cat process1.c
#include <stdio.h>
#include <unistd.h>
int main()
{
printf("This is second lab");
fork();
printf("Process is creating");
printf(" I am process %d", getpid());
return 0;
}
ankit@ankit-virtual-machine:~/lab02$ ls
process1.c
ankit@ankit-virtual-machine:~/lab02$ gcc -o process1 process1.c
ankit@ankit-virtual-machine:~/lab02$ ./process1
This is second labProcess is creating I am process 7237This is second labProcess I am process 7237ankit@ankit-virtual-machine:~/lab02$
```

```
ankit@ankit-virtual-machine:~$ ls
ankit  Desktop  Downloads  Music  Public  Templates
ankit.txt  Documents  lab02  Pictures  snap  Videos
ankit@ankit-virtual-machine:~$ cd lab02
ankit@ankit-virtual-machine:~/lab02$ touch process2.c
ankit@ankit-virtual-machine:~/lab02$ nano process2.c
ankit@ankit-virtual-machine:~/lab02$ cat process2.c
//create a child and parent process
#include <stdio.h>
#include <unistd.h>
int main()
{
int pid;
printf("This is our second lab");
printf("I am the parent process and pid is %d", getpid());
printf("before use of forking");
printf("after forking");
if (pid == 0)
printf(" I am the child process and pid is %d ", getpid());
else
printf(" I am the parent process and pid %d", getpid());

return 0;
}
ankit@ankit-virtual-machine:~/lab02$ gcc -o process2 process2.c
ankit@ankit-virtual-machine:~/lab02$ ./process2
This is our second labI am the parent process and pid is 7293before use of forkingafter forking I am the parent process and pid 7293ankit@ankit-virtual-machine:~/lab02$
```

```
ankit@ankit-virtual-machine:~$ ls
ankit  Desktop  Downloads  Music  Public  Templates
ankit.txt  Documents  lab02  Pictures  snap  Videos
ankit@ankit-virtual-machine:~$ cd lab02
ankit@ankit-virtual-machine:~/lab02$ touch process3.c
ankit@ankit-virtual-machine:~/lab02$ nano process3.c
ankit@ankit-virtual-machine:~/lab02$ cat process3.c
//to use multiple fork statement
#include<stdio.h>
#include<unistd.h>
int main() {
printf(" before first forking statement");
fork();
printf("After forking statement");
fork();
printf(" After second forking");
printf(" hello world form process %d", getpid());
return 0;
}
ankit@ankit-virtual-machine:~/lab02$ gcc -o process3 process3.c
ankit@ankit-virtual-machine:~/lab02$ ls
process1  process1.c  process2  process2.c  process3  process3.c
ankit@ankit-virtual-machine:~/lab02$ ./process3
before first forking statementAfter forking statement After second forking hello world form process 7448 before first forking statementAfter forking statement After
second forking hello world form process 7449 before first forking statementAfter forking statement After second forking hello world form process 7451 before first f
ankit@ankit-virtual-machine:~/lab02$ █
```

CONCLUSION

HENCE WE STUDIED SOME PROCESS MANAGEMENT COMMANDS AND IMPLEMENTED THEM TO CREATE A PROCESS WITH PID, CREATED CHILD AND PARENT PROCESSES AND USED MULTIPLE FORK STATEMENT.